ADP470S

Applications Development Practice 4

TERM 1 — MASTER STUDY GUIDE

Data Structures & Algorithms | NQF Level 7 | CPUT | 2026

Topics Covered	Assessment Coverage
Recursion vs Iteration	Test 1 (2024) — Q1 & Q2
Big O Notation & Complexity	Test 1 (2025) — Q1
Linear & Binary Search	Practice Questions
Bubble Sort	2025 Q3.1-3.2
Quick Sort	2024 Q2.1 & 2025 Q3.3
Heap Sort	2025 Q3.5

1 What is an Algorithm?

Definition

An algorithm is a finite, ordered set of well-defined steps that solves a problem.

It must be: Finite · Definite · Input/Output · Effective · Correct

1.1 Big O Notation — Measuring Performance

Big O notation describes how the running time (or memory) of an algorithm grows as the input size n grows. It gives us the worst-case upper bound.

Notation	Name	Example	Speed
`O(1)`	Constant	Array index lookup	⚡ Fastest
`O(log n)`	Logarithmic	Binary Search	🚀 Very Fast
`O(n)`	Linear	Linear Search / Loop	✅ OK
`O(n log n)`	Linearithmic	Merge/Heap Sort	🔶 Good
`O(n²)`	Quadratic	Bubble Sort (worst)	🐢 Slow
`O(2ⁿ)`	Exponential	Naive recursion	🐌 Very Slow

1.2 Big O Code Examples (From 2025 Test — Q1)

Example A

```
int sum = 0;
for (int i = 0; i < n; i++) {     // Loop 1: runs n times
    sum = sum + a[i];
}
for (int j = 0; j < n; j++) {     // Loop 2: runs n times
    sum = sum + a[j];
}
```

Answer: O(n)

Loop 1 runs n times → O(n)

Loop 2 runs n times → O(n)

Total = O(n) + O(n) = O(2n)

Drop the constant: O(n) ← FINAL ANSWER

Rule: In Big O, constants are dropped. O(2n) simplifies to O(n).

Example B

```
int prod = 1;
for (int i = 1; i < n; i++) {          // Outer loop: runs n times
    for (int j = i; j < n; j = j*3) { // Inner loop: j triples each time
        prod = prod * j;               //   → runs log₃(n) times
    }
}
```

Answer: O(n log n)

Outer loop runs n times → O(n)

Inner loop: j = i, i*3, i*9 ... i.e. j multiplied by 3 each step

→ Inner loop runs $\log_3(n)$ times per outer iteration

Total = O(n) × O(log n) = O(n log n)

Rule: Nested loops multiply. Tripling j each step = logarithmic.

Example C

```
int sum = 0;
for (i = 0; i <= n-1; i += 3) {    // Outer: i goes 0,3,6,... → n/3 steps
    for (j = n-1; j > 0; j--) {    // Inner: j goes n-1 down to 1 → n steps
        sum = sum + (i * j);
    }
}
```

Answer: O(n²)

Outer loop: i increments by 3 each time → runs n/3 times

Inner loop: j decrements by 1 each time → runs n times

Total = O(n/3) × O(n) = O(n²/3)

Drop the constant 1/3: → O(n²) ← FINAL ANSWER

Rule: Even if outer loop increments by 3, constants are dropped.

2 Recursion vs Iteration

Key Concept
Recursion: A method that calls ITSELF to solve a smaller version of the same problem.
Every recursive solution must have: (1) A BASE CASE (2) A RECURSIVE CASE
Think of it like Russian dolls — each doll opens to reveal a smaller doll inside.

2.1 Factorial — The Classic Example

Mathematical definition: **n! = 1 (if n = 0 or n = 1)** | **n! = n × (n-1)! (if n ≥ 2)**

2.1A Recursive Java Implementation

```
public static int factorialRecursive(int n) {
    if (n == 0 || n == 1) {        // BASE CASE: stop here
        return 1;
    }
    return n * factorialRecursive(n - 1); // RECURSIVE CASE
}
```

2.1B Iterative Java Implementation

```
public static int factorialIterative(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) { // Loop from 2 to n
        result = result * i;
    }
    return result;
}
```

2.2 Recursive Trace for factorial(5) — (2024 Test Q1.1 / 2025 Q2.1)

WINDING UP (calls building)	UNWINDING (returning values)
factorial(5) → calls factorial(4)	factorial(1) returns 1
factorial(4) → calls factorial(3)	factorial(2) = 2×1 = 2
factorial(3) → calls factorial(2)	factorial(3) = 3×2 = 6
factorial(2) → calls factorial(1)	factorial(4) = 4×6 = 24

`factorial(1) → BASE CASE! returns 1`	`factorial(5) = 5×24 = 120 ✓`

```
┌──────────────────────────────────┐  ← TOP of stack (last called)
│  factorial(1)   returns 1        │
├──────────────────────────────────┤
│  factorial(2)   returns 2×1=2    │
├──────────────────────────────────┤
│  factorial(3)   returns 3×2=6    │
├──────────────────────────────────┤
│  factorial(4)   returns 4×6=24   │
├──────────────────────────────────┤
│  factorial(5)   returns 5×24=120 │  ← BOTTOM (first called)
└──────────────────────────────────┘
```

2.3 Performance: Recursion vs Iteration (2024 Q1.3 / 2025 Q2.2)

	RECURSIVE	ITERATIVE
Time Complexity	O(n) — n recursive calls	O(n) — n loop iterations
Space Complexity	O(n) — n stack frames!	O(1) — only one variable
Stack Risk	Can cause StackOverflow for large n	No stack risk
Readability	Clean & matches maths formula	More verbose
Real World Pref.	❌ Avoid for simple factorial	✅ Preferred for large n

Exam Tip: Space Complexity

Recursive factorial = O(n) space because each call creates a NEW stack frame in memory.

When n=1000, you have 1000 stack frames open simultaneously!

Iterative = O(1) space — only the "result" variable exists, no matter how large n is.

CONCLUSION: For simple factorial, ITERATIVE is preferable in real world.

3 Searching Algorithms

3.1 Linear Search — O(n)

How it works
Go through the array ONE element at a time from the start.
Compare each element to the target value. If found: return the index. If end reached: return -1 (not found).

Step	Action	Array State
1	Check index 0: 90 == 109? No	[90, 23, 5, 109, 12]
2	Check index 1: 23 == 109? No	[90, 23, 5, 109, 12]
3	Check index 2: 5 == 109? No	[90, 23, 5, 109, 12]
4	Check index 3: 109 == 109? YES! Found at index 3	[90, 23, 5, 109✓, 12]

Best Case	Average Case	Worst Case
O(1)	O(n)	O(n)
Target is 1st element	Target in middle on avg	Target is last or not found

3.2 Binary Search — O(log n)

Requirement
Array MUST be SORTED first! Binary search only works on sorted arrays.
Works like a phone book: open middle, go left or right based on comparison.

Step	Action	Array State
1	low=0,high=4 → mid=2 → arr[2]=23 == 23? YES! Found!	[5, 12, 23✓, 90, 109]

Step	Action	Array State
1	low=0,high=4 → mid=2 → arr[2]=23 < 90 → go RIGHT	[5, 12, 23, 90, 109]

2	low=3,high=4 → mid=3 → arr[3]=90 == 90? YES! Found at index 3	[5, 12, 23, 90✓, 109]

Best Case	Average Case	Worst Case
O(1)	O(log n)	O(log n)

4 Bubble Sort

How it works
Compare adjacent elements. If left > right, SWAP them. Repeat passes through the array until no swaps occur.
After each pass, the LARGEST unsorted element "bubbles up" to its correct position.
Array: {90, 23, 5, 109, 12} — worked from 2025 Test Q3.1

4.1 Step-by-Step Trace: {90, 23, 5, 109, 12}

Step	Action	Array State
1.1	Compare 90 & 23 → 90>23 → SWAP	[23, 90, 5, 109, 12]
1.2	Compare 90 & 5 → 90>5 → SWAP	[23, 5, 90, 109, 12]
1.3	Compare 90 & 109 → 90<109 → No swap	[23, 5, 90, 109, 12]
1.4	Compare 109 & 12 → 109>12 → SWAP → 109 in place ✓	[23, 5, 90, 12, 109]

Step	Action	Array State
2.1	Compare 23 & 5 → 23>5 → SWAP	[5, 23, 90, 12, 109]
2.2	Compare 23 & 90 → 23<90 → No swap	[5, 23, 90, 12, 109]
2.3	Compare 90 & 12 → 90>12 → SWAP → 90 in place ✓	[5, 23, 12, 90, 109]

Step	Action	Array State
3.1	Compare 5 & 23 → 5<23 → No swap	[5, 23, 12, 90, 109]
3.2	Compare 23 & 12 → 23>12 → SWAP → 23 in place ✓	[5, 12, 23, 90, 109]

Step	Action	Array State

4.1	Compare 5 & 12 → 5<12 → No swap → SORTED! ✓	`[5, 12, 23, 90, 109]`

5	12	23	90	109
0	1	2	3	4

☑ Final sorted array: {5, 12, 23, 90, 109}

4.2 Bubble Sort Complexity (2025 Q3.2)

Best Case	Average Case	Worst Case
O(n)	**O(n²)**	**O(n²)**
Already sorted (with flag)	Random order	Reverse sorted: {5,4,3,2,1}

Why O(n²)?
Two nested loops: outer runs n times, inner runs n times.
Total comparisons ≈ n × n = n²
Example: n=5 → up to 25 comparisons worst case.

5 Quick Sort

How it works — Divide & Conquer
1. Choose a PIVOT element (often last, first, or middle element).
2. PARTITION: put all elements < pivot to the LEFT, all > pivot to the RIGHT.
3. Recursively apply the same process to left and right sub-arrays.
4. When sub-arrays have 0 or 1 element, array is sorted!

5.1 Step-by-Step Trace: {90, 23, 5, 109, 12} (2025 Test Q3.3)

90	23	5	109	**12**
0	1	2	3	4

Initial array. Pivot = 12 (last element, shown in orange)

Step	Action	Array State
R1	Full array: pivot=12. Elements < 12: {5}. Elements > 12: {90,23,109}.	`[5, 12, 23, 90, 109]`
R2	Left sub-array {5}: only 1 element → already sorted ✓	`[5]`
R3	Right {90,23,109}: pivot=109. < 109: {90,23}. > 109: {}	`[90, 23, 109]`
R4	Sub {90,23}: pivot=23. < 23: {}. > 23: {90}	`[23, 90]`
R5	Merge all: 5, 12, 23, 90, 109 ✓	`[5, 12, 23, 90, 109]`

5	12	23	90	109
0	1	2	3	4

☑ Final sorted array

5.2 Detailed Partition Trace: {7, 5, 15, 20, 16, 17, 8, 3} (2024 Test Q2.1)

Step	Action	Array State
R1	pivot=3 (last). All others > 3 → {} | 3 | {7,5,15,20,16,17,8}	`[3, 5, 7, 8, 15, 16, 17, 20]`

R2	Right sub {7,5,15,20,16,17,8}: pivot=8	`[3, 5, 7, 8, 15, 16, 17, 20]`
R3	< 8: {7,5} | 8 | > 8: {15,20,16,17}	`[3, 5, 7, 8, 15, 16, 17, 20]`
R4	Sub {7,5}: pivot=5 → {}/5/{7}	`[3, 5, 7, 8, 15, 16, 17, 20]`
R5	Sub {15,20,16,17}: pivot=17 → {15,16}/17/{20}	`[3, 5, 7, 8, 15, 16, 17, 20]`
R6	All single elements → DONE ✓	`[3, 5, 7, 8, 15, 16, 17, 20]`

5.3 Quick Sort Complexity (2025 Q3.4 / 2024 Q2.2/2.4/2.6)

Best Case	Average Case	Worst Case
O(n log n)	**O(n log n)**	**O(n^2)**
Pivot always splits array in half	Random/typical data	Pivot is always smallest/largest (sorted input)

6 Heap Sort

Key Concept: What is a Heap?
A heap is a BINARY TREE with one special property:
MAX-HEAP: Parent is always GREATER THAN or EQUAL TO its children.
We first convert the array into a Max-Heap (heapify), then extract the max repeatedly.
Think of it like a tournament bracket — the best player always rises to the top!

6.1 How to Read an Array as a Binary Tree

Node at index i	Left child	Right child
`arr[i]`	`arr[2i + 1]`	`arr[2i + 2]`

6.2 Step-by-Step Trace: {90, 23, 5, 109, 12} (2025 Test Q3.5)

```
Initial array:  [90, 23, 5, 109, 12]
                  90
                /    \
              23      5
             /  \
           109   12

Start heapifying from last non-leaf node (index 1 = value 23):
  Node 23 has children 109 and 12.
  109 > 23 → SWAP 23 and 109

After first heapify:  [90, 109, 5, 23, 12]
                  90
                /    \
             109      5
             /  \
           23   12
```

```
Now heapify root (index 0 = value 90):
  Node 90 has children 109 and 5.
  109 > 90 → SWAP 90 and 109


After second heapify:  [109, 90, 5, 23, 12]
                 109      ← MAX at root ✓
                /    \
              90       5
             /  \
           23   12


Max-Heap built! ✓
```

Step	Action	Array State
E1	Swap root(109) with last element(12). Remove 109 → sorted region: [109]	`[12, 90, 5, 23, 109]`
E2	Re-heapify: 12 at root, swap with 90	`[90, 12, 5, 23, 109]`
E3	Swap root(90) with last unsorted(23). Remove 90 → [90,109]	`[23, 12, 5, 90, 109]`
E4	Re-heapify: 23 at root, swap with 12... Extract 23 → [23,90,109]	`[12, 5, 23, 90, 109]`
E5	Extract 12 → [12,23,90,109]	`[5, 12, 23, 90, 109]`
E6	Last element 5 in place → SORTED ✓	`[5, 12, 23, 90, 109]`

6.3 Heap Sort Complexity (2025 Q3.6 / 2024 Q2.3/2.5/2.7)

Best Case	Average Case	Worst Case
O(n log n)	**O(n log n)**	**O(n log n)**
ALL CASES same! (unlike QuickSort)	Consistent guaranteed performance	Never degrades to O(n²)

Heap vs Quick Sort — Key Exam Fact
HeapSort is ALWAYS O(n log n) — best, average, and worst case.

QuickSort is $O(n^2)$ in the worst case (bad pivot choice).
HeapSort uses O(1) extra space. QuickSort uses O(log n) for the recursion stack. In practice, QuickSort is often faster because of cache efficiency, but HeapSort is safer.

7 Algorithm Comparison Cheat Sheet

Algorithm	Best	Average	Worst	Space	Stable
Linear Search	O(1)	O(n)	O(n)	O(1)	—
Binary Search	O(1)	O(log n)	O(log n)	O(1)	—
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	✅ Yes
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	✅ Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	❌ No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	❌ No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	✅ Yes

8 Worked Test Papers — Full Solutions

ADP470S IN-CLASS TEST — 7 MARCH 2024 (Full Worked Solutions)

2024 Q1.1 — Recursive Factorial Trace for n=5

Question
Using a trace, explain how the recursive factorial method works for n=5. [5 marks]

```
Base case: if n == 0 or n == 1, return 1.

Trace of factorial(5):
  factorial(5) = 5 × factorial(4)
    factorial(4) = 4 × factorial(3)
      factorial(3) = 3 × factorial(2)
        factorial(2) = 2 × factorial(1)
          factorial(1) = 1  ← BASE CASE, return 1
        factorial(2) = 2 × 1 = 2  (returns 2)
      factorial(3) = 3 × 2 = 6  (returns 6)
    factorial(4) = 4 × 6 = 24  (returns 24)
  factorial(5) = 5 × 24 = 120  (returns 120)

Result: 5! = 120 ✓
```

Marking Guide (5 marks)
1 mark: Correct base case identified (n=0 or n=1 → return 1)
1 mark: Correct recursive call shown (n × factorial(n-1))
1 mark: Winding up phase shown (each call leads to next)
1 mark: Unwinding phase shown (values returned upward)
1 mark: Correct final answer 120

2024 Q1.2 — Iterative Factorial Trace for n=5

Question

Using a trace, explain how the iterative factorial method works. [5 marks]

Iteration	i value	result = result × i	result value
Start	—	`result = 1`	1
i=2	2	`result = 1 × 2`	2
i=3	3	`result = 2 × 3`	6
i=4	4	`result = 6 × 4`	24
i=5	5	`result = 24 × 5`	**120 ← FINAL**

2024 Q1.3 — Big O Comparison: Recursive vs Iterative Factorial

Question

Using Big O notation, compare the performance from a viewpoint of time and space utilisation. [6 marks]

```
RECURSIVE FACTORIAL:
  Time Complexity:  O(n)
    → Makes exactly n recursive calls before hitting base case.
    → Each call does O(1) work (one multiplication).
    → Total: n × O(1) = O(n)

  Space Complexity: O(n)
    → Each recursive call creates a NEW STACK FRAME in memory.
    → n=5 → 5 stack frames open simultaneously.
    → Stack holds: return address, parameter, local variables per frame.
    → Risk: StackOverflowError if n is very large.

ITERATIVE FACTORIAL:
  Time Complexity:  O(n)
    → Loop runs from i=2 to i=n → n-1 iterations ≈ O(n)

  Space Complexity: O(1)  ← CONSTANT
```

```
    → Only stores ONE variable (result) regardless of n.
    → No stack frames created.
```

2024 Q1.4 — Which is Preferable in Real World?

Question
Which one is preferable in the real world? [4 marks]

```
ITERATIVE is preferable for computing factorial in the real world.

Reasons:
1. Space efficiency: O(1) vs O(n) — no risk of StackOverflowError.
2. Performance: No overhead of function calls and stack frame creation.
3. Scalability: Works for very large n without running out of stack memory.
4. Simplicity: Easy to understand and debug.

NOTE: Recursion is preferable in cases where the problem is naturally
recursive (e.g., tree traversals, divide-and-conquer) and n is bounded.
For simple factorial, there is no benefit to using recursion.
```

ADP470S TEST 1 — 3 APRIL 2025 (75 Marks) — Full Worked Solutions

2025 Q1 — Determine Big O for Code Excerpts

2025 Q2.1 — Recursive Factorial Execution Trace

Question
Given factorial = 1 if n=0 or n=1, else n×factorial(n-1). Demonstrate how recursive code executes. [marks]

```
Java code:
public static int factorial(int n) {
    if (n == 0 || n == 1) return 1;     // Base case
    return n * factorial(n - 1);         // Recursive case
}

Execution trace for factorial(4):
  factorial(4) calls factorial(3)
    factorial(3) calls factorial(2)
      factorial(2) calls factorial(1)
        factorial(1) = 1  ← BASE CASE
      factorial(2) = 2 × 1 = 2
    factorial(3) = 3 × 2 = 6
  factorial(4) = 4 × 6 = 24

Result: 4! = 24 ✓
```

2025 Q2.2 — Space and Time Complexity of Factorial

	RECURSIVE	ITERATIVE
Time	O(n)	O(n)

Space	**O(n) — n stack frames**	**O(1) — one variable**

2025 Q3.1 — Bubble Sort on {90, 23, 5, 109, 12}

Quick Answer Summary
Pass 1: {23,5,90,12,109} — 109 placed
Pass 2: {5,23,12,90,109} — 90 placed
Pass 3: {5,12,23,90,109} — 23 placed
Pass 4: No swaps needed — SORTED! {5,12,23,90,109}

2025 Q3.2 — Big O for Bubble Sort

```
Bubble Sort has two nested loops:
  Outer loop:  runs n-1 times (n = number of elements)
  Inner loop:  runs n-1-i times per outer iteration

Total comparisons = (n-1) + (n-2) + ... + 1 = n(n-1)/2
                  = (n² - n) / 2
                  ≈ O(n²)     (drop lower terms and constants)

Best case:    O(n)  — if already sorted (with swapped flag optimization)
Average case: O(n²)
Worst case:   O(n²) — reverse sorted array: {12,109,90,23,5}
```

2025 Q3.3 — Quick Sort on {90, 23, 5, 109, 12}

Quick Answer Summary
Pivot = 12 (last element). Partition: {5} | 12 | {90,23,109}
Sub {90,23,109}: Pivot=109. Partition: {90,23} | 109 | {}
Sub {90,23}: Pivot=23. Partition: {} | 23 | {90}
Combine: {5, 12, 23, 90, 109} ✓

2025 Q3.4 — Big O for Quick Sort

```
Best Case: O(n log n)
  → Pivot always splits array into two EQUAL halves.
  → log n levels of recursion, n work per level → O(n log n)

Average Case: O(n log n)
  → Random pivot gives roughly balanced splits on average.

Worst Case: O(n²)
  → Pivot is always smallest or largest element.
  → One side gets 0 elements, other gets n-1.
  → Happens when array is already sorted and pivot = last element.
  → n levels of recursion, n work per level → O(n²)
```

2025 Q3.5 — Heap Sort on {90, 23, 5, 109, 12}

Quick Answer Summary
Phase 1 — Build Max-Heap: {109, 90, 5, 23, 12}
Phase 2 — Extract max repeatedly:
Extract 109 → {90, 23, 5, 12, | 109}
Extract 90 → {23, 12, 5 | 90, 109}
Extract 23 → {12, 5 | 23, 90, 109}
Extract 12 → {5 | 12, 23, 90, 109}
SORTED: {5, 12, 23, 90, 109} ✓

2025 Q3.6 — Big O for Heap Sort

```
Heap Sort is ALWAYS O(n log n) — best, average, AND worst case.

Why?
  Phase 1 (Build Max-Heap):    O(n)
  Phase 2 (n extractions):     Each extraction = O(log n)
```

```
                                n extractions × O(log n) = O(n log n)

```

```
Total: O(n) + O(n log n) = O(n log n)
```

```
Space Complexity: O(1) — in-place sorting, no extra array needed.
```

```
Key advantage over QuickSort: No worst-case O(n²) scenario.
```

9 Quick-Reference Exam Card

Things to ALWAYS write in your exam:
1. Big O — state the complexity AND explain why (show the loops or call count)
2. For recursion — show BASE CASE and RECURSIVE CASE explicitly
3. For traces — show EVERY step, including "no swap" steps for bubble sort
4. For heap — show both phases: (1) Build Heap then (2) Extract
5. For quick sort — show the pivot chosen AND the partition each step

Common Exam Trap	Correct Answer
❌ "Recursion is always O(1) space"	☑ Recursion is O(n) space — n stack frames!
❌ "O(2n) is different from O(n)"	☑ O(2n) = O(n). Constants are dropped in Big O.
❌ "Binary search on unsorted array"	☑ Binary search REQUIRES a sorted array first.
❌ "HeapSort worst case is $O(n^2)$"	☑ HeapSort is ALWAYS O(n log n) in all cases.
❌ "QuickSort is always O(n log n)"	☑ QuickSort worst case is $O(n^2)$ with bad pivot.
❌ "Bubble sort best case is $O(n^2)$"	☑ Best case is O(n) if already sorted (with flag).

Good luck on Thursday! You have got this, Thabiso | ADP470S 2026